
Handwritten Mathematical Expressions to LaTeX

Jessie Guo

jessiejiaxin.guo@mail.utoronto.ca

https://github.com/JessieGjx/APS360_img_to_latex

Introduction

Mathematical expressions are often derived by hand before being incorporated into documents. Manually converting these expressions into LaTeX is time-consuming and error-prone. Unlike normal text recognition, these expressions contain complex two-dimensional structures and large symbol sets. This project develops a deep learning model that converts handwritten mathematical expression images into LaTeX sequences, reducing the effort required to digitize handwritten mathematics for technical and educational applications. The model uses a convolutional neural network (CNN) encoder to extract visual features and a transformer decoder to generate the corresponding LaTeX tokens. Deep learning is suitable for this task because it can learn both visual patterns from handwritten symbols and the structural relationships required to generate valid LaTeX output.

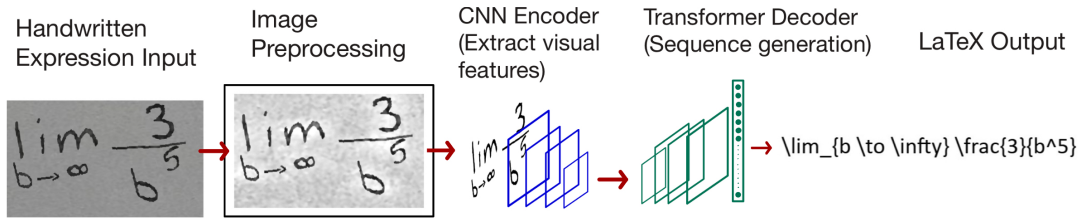


Figure 1: Overview of the handwritten mathematical expressions to LaTeX model pipeline.

Background & Related Work

Prior work in handwritten mathematical expression recognition has progressed from classical OCR systems to end-to-end neural image-to-sequence models. Deng et al. (1) introduced a CNN encoder with an attention-based decoder for image-to-LaTeX generation, establishing a foundational encoder-decoder framework. Gurgurov and Morshnev (2) demonstrated the effectiveness of Transformer-based decoders with pretraining and handwritten fine-tuning, motivating the training strategy adopted in this project. Peng et al. (3) incorporated graph neural networks to improve structural reasoning between mathematical symbols. Suzuki et al. (4) proposed the classical INFTY OCR system, representing an earlier segmentation-based approach for mathematical expression recognition. Finally, Xie et al. (5) established the ICDAR 2023 CROHME benchmark, providing Bilingual Evaluation Understudy (BLEU) score-based evaluation metrics for handwritten mathematical expression recognition. Motivated by these works, this project investigates a CNN-Transformer encoder-decoder architecture for handwritten image-to-LaTeX generation.

Data Processing

The IM2LATEX-100K (6) dataset from Kaggle is used for pretraining, providing large-scale rendered image-LaTeX pairs to learn mathematical syntax and structural relationships. The pretrained model is then fine-tuned on the handwritten HME100K dataset (7) from Kaggle to adapt to handwriting variations. A separate self-collected dataset of undergraduate engineering equations is reserved exclusively for evaluating generalization to unseen handwriting styles.

Data Cleaning Process: IM2LATEX-100K samples with missing labels or images, corrupted files, duplicates, blank images, or formulas exceeding 500 tokens were removed. Since the images are rendered, only validation and aspect-ratio-preserving padding were applied. For HME100K and the self-collected dataset, invalid samples and non-mathematical characters were removed. Images were converted to grayscale, denoised using fast non-local means, enhanced with CLAHE contrast adjustment, normalized,

and padded to preserve handwriting strokes and aspect ratio while reducing background variation. LaTeX labels were cleaned and tokenized using whitespace-based splitting, with a shared 636-token vocabulary created from IM2LATEX-100K and HME100K.

Data statistics: IM2LATEX-100K and HME100K use an 80/10/10 split for training, validation, and testing. After cleaning, IM2LATEX-100K contains 90,391 samples (average 56.9 tokens, aspect ratio 5.83), while HME100K contains 99,109 handwritten expressions (average 17.6 tokens, aspect ratio 4.47) with variation in brightness and contrast, which motivates the use of contrast enhancement and background normalization. The self-collected test set contains 20 expressions (average 31.7 tokens, aspect ratio 3.94). The final tokenizer contains 636 unique LaTeX tokens.

Preprocessing Example: Three enhancement methods were evaluated for HME100K. Thresholding improved separation but introduced artifacts and removed faint strokes, while CLAHE preserved handwriting details and was selected for CNN feature extraction (Figure 2).

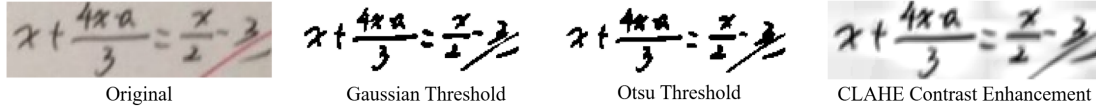


Figure 2: Comparison of preprocessing methods on HME100K sample.

Primary Model: CNN + Transformer

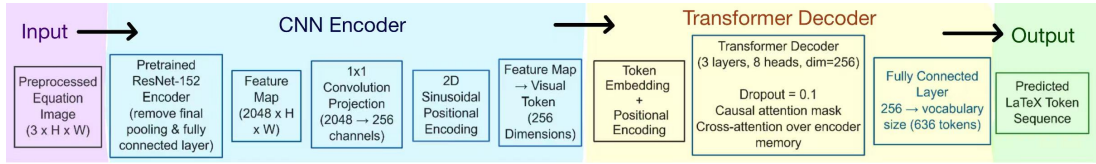


Figure 3: The primary CNN-Transformer model. Model details are shown in the figure

The primary model is a ResNet-Transformer encoder-decoder architecture. The encoder uses an ImageNet-initialized ResNet-152 backbone with the original convolutional blocks, kernel sizes, skip connections, batch normalization, and ReLU activations. The final classification layers are removed, and the extracted visual features are projected from 2048 channels to a 256-dimensional embedding space before being converted into spatial visual tokens. The decoder generates LaTeX sequences autoregressively using a Transformer decoder. Previous LaTeX tokens are embedded with learned positional information, and causal masking prevents access to future tokens during generation. The decoder uses encoder visual tokens through cross-attention to combine image features with generated sequence context. A final fully connected layer converts decoder outputs into vocabulary probabilities over the 636 LaTeX tokens.

Training Configuration: The model was trained using staged pretraining and fine-tuning. First, it was pretrained on IM2LATEX-100K for 140 epochs to learn LaTeX syntax and mathematical structures. For the first 100 epochs, the ResNet-152 encoder was frozen, and only the decoder and projection layers were trained using AdamW (learning rate: 2×10^{-5} , batch size: 24). The best checkpoint was then trained for 40 additional epochs with the final ResNet layer (layer 4) unfrozen to improve visual feature adaptation. The pretrained model was fine-tuned on HME100K for 30 epochs with the final ResNet layer unfrozen. The optimizer was reset using AdamW (learning rate: 1×10^{-5} , weight decay: 1×10^{-4} , batch size: 24) with a ReduceLROnPlateau scheduler (factor: 0.5, minimum learning rate: 1×10^{-7}). Cross-entropy loss with label smoothing of 0.03 was used for training. This staged strategy enables learning general LaTeX structures from rendered expressions before adapting to handwritten inputs.

Baseline model: CNN + RNN

A lightweight CNN-RNN model was implemented as the baseline. The CNN encoder extracts visual features from formula images, which are then used to initialize a recurrent decoder for auto-regressive LaTeX generation. Unlike the primary Transformer model, the baseline does not use attention mechanisms or positional encoding, providing a simpler reference for evaluating the benefit of Transformer-based sequence modeling. The baseline was trained on IM2LaTeX-100K using Adam optimizer with a batch size of 64, learning rate of 1×10^{-3} , and 15 epochs.

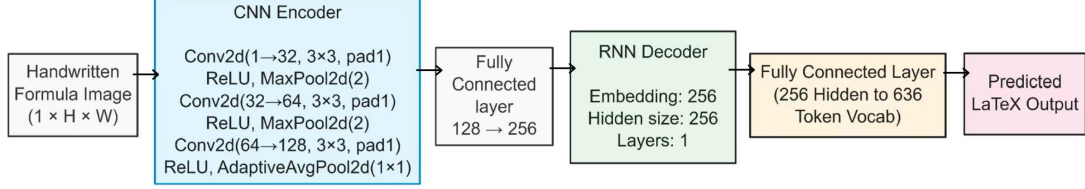


Figure 4: Baseline CNN-RNN encoder-decoder architecture.

Quantitative Results

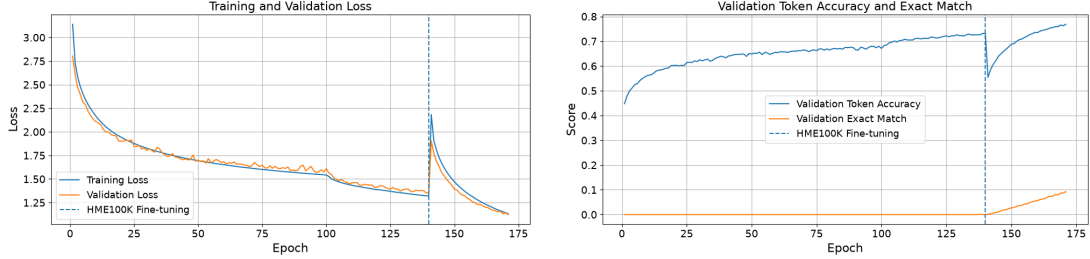


Figure 5: Training curves for the primary CNN-Transformer model. At epoch 100, the encoder was partially unfrozen, leading to further loss reduction after the initial plateau as visual features adapted to the task. The sudden change at epoch 140 corresponds to switching datasets for fine-tuning, causing a temporary performance drop due to the domain shift from rendered to handwritten expressions.

Table 1: Performance comparison between the baseline RNN and primary Transformer-based model using beam search (beam size = 3) on three test sets. BLEU scores are reported at token level and after whitespace normalization to evaluate sequence similarity, while exact match accuracy measures complete LaTeX sequence correctness.

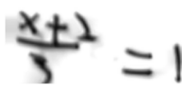
Dataset	Model	BLEU (tok)	BLEU (norm)	Exact Match
IM2LaTeX-100K test subset n=9447	Baseline RNN	7.22	10.17	0.00%
	Transformer	20.47	27.94	0.00%
HME100K test subset n=9911	Baseline RNN	3.04	4.56	0.00%
	Transformer	42.30	50.58	10.68%
Self-Collected n=20	Baseline RNN	4.90	7.00	0.00%
	Transformer	13.32	19.78	0.00%

The primary model consistently outperformed the baseline across all datasets. The largest improvement was observed on HME100K, where normalized BLEU increased from 4.56 to 50.58 and exact match improved to 10.68%. The lower performance on IM2LaTeX-100K is expected because the reported model is the final HME100K fine-tuned checkpoint rather than the original IM2LaTeX-only weights. Additionally, IM2LaTeX expressions contain longer and more complex LaTeX sequences, increasing the difficulty of exact sequence prediction. Quantitative results for the self-collected dataset will be discussed in Model Performance on New Data section.

Qualitative Results

Two examples were selected to show that the model effectively captures high-level mathematical structure and formatting while performing better on handwritten expressions than on longer rendered formulas. Errors are mainly concentrated in symbol recognition. Generalization to unseen handwriting styles is evaluated in later sections using the self-collected dataset.

As shown in Figure 6, the left example is a near-correct HME100K prediction demonstrating the model’s ability to recover handwritten expressions, with only a minor symbol substitution ($2 \rightarrow 3$), while the right example shows a common IM2LaTeX-100K failure case where the overall LaTeX structure is preserved but variables and specialized operators are incorrectly recognized.

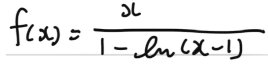


	Ground Truth $\frac{x+2}{3} = 1$	Input (IM2LaTeX-100K)	$\hat{H} \equiv \sum_{\alpha=1}^{N+1} c_{\alpha} a_{\alpha}^{\dagger} a_{\alpha}$	Prediction (rendered)
Input (HME100k)	Prediction: $\frac{x+3}{3} = 1$	$\hat{H} \equiv \sum_{\alpha=1}^{N+1} c_{\alpha} a_{\alpha}^{\dagger} a_{\alpha}$	$H_i \equiv \sum_{i=1}^N p_i^2 \sin^2 \theta_i \sin^2 \theta_i$	$H_i \equiv \sum_{i=1}^N p_i^2 \sin^2 \theta_i \sin^2 \theta_i$

Figure 6: Representative predictions from the test subsets from the primary model.

Model Performance on New Data

To evaluate generalization beyond the training distribution, the final HME100K fine-tuned Transformer was tested on a self-collected dataset of 20 handwritten mathematical expressions. These samples were collected separately and were not used during training, validation, or hyperparameter tuning. Using beam search inference (beam size = 3), the model achieved a normalized BLEU score of 19.78 and an exact match accuracy of 0.00%. Compared with the HME100K test performance (42.30 BLEU score and 10.68% exact match accuracy), the decrease reflects the domain gap between benchmark handwritten expressions and unseen user-generated handwriting. The 0.00% exact match is expected for a small dataset with longer expressions, where a single token error causes the sequence to be incorrect.



	Ground Truth: $f(x) = \frac{x}{1 - \ln(x-1)}$	$f'(x) = \frac{e^x - 1}{e^x - e^x}$	Prediction: $f'(x) = \frac{e^x - 1}{e^x - e^x}$
--	--	-------------------------------------	--

Figure 7: Example prediction on the self-collected dataset. The model preserves the overall equation structure but makes symbol-level recognition errors. In this example, the handwritten $\ln$ operator is interpreted as an exponential expression, likely due to the cursive “l” resembling an “e”, while the unconventional handwritten “x” style introduces additional visual ambiguity.

Figure 7 shows that the model successfully learns high-level LaTeX structure and equation formatting, while remaining sensitive to unseen handwriting variations. The reduced performance is expected due to the domain gap from HME100K, the small evaluation size of 20 samples, and the lack of training on personal handwriting styles.

Discussion

An important observation during development was that direct training on the combined IM2LATEX-100K and HME100K datasets resulted in poor generation quality. The decoder produced syntactically valid LaTeX but often ignored image-specific information and collapsed toward frequent token patterns (Appendix A). This suggested that the Transformer decoder learned LaTeX sequence statistics faster than the CNN encoder learned meaningful visual representations. To determine whether this failure was due to insufficient model capacity, an overfitting experiment was performed. The model achieved near-perfect token accuracy and high exact match on a small training set (Appendix B), confirming that the CNN-Transformer architecture was capable of learning the image-to-LaTeX mapping. This was a key insight during development, indicating that the main limitation was the training strategy rather than the architecture itself. This motivated the final staged training approach, where the model first learned LaTeX grammar from IM2LaTeX before adapting visual representations to handwritten expressions through HME100K fine-tuning. Results demonstrate that transfer learning from rendered to handwritten mathematical expressions is effective. Remaining errors are primarily caused by visually ambiguous symbols and unseen handwriting styles. Future improvements include training on a larger and more diverse handwriting dataset, stronger image augmentation, and longer HME100K fine-tuning, as validation loss had not fully converged after 30 epochs. Further training was not performed due to course time and computational constraints.

Ethical Considerations

The datasets used may contain bias in handwriting styles and mathematical domains, as they primarily reflect the types of expressions represented in the training data. As a result, the model may not generalize well to less frequent mathematical symbols, specialized domains, or uncommon handwriting styles such as cursive writing. Although HME100K improves realism by including real handwritten expressions, it may still underrepresent the diversity of real-world handwriting and mathematical notation. The datasets are publicly available and anonymized, and no personal or sensitive information is included. The model is intended as an assistive tool rather than a replacement for human verification, as incorrect LaTeX generation could lead to misinterpretation of mathematical expressions.

References

- [1] Y. Deng, A. Kanervisto, J. Ling, and A. M. Rush, “Image-to-markup generation with coarse-to-fine attention,” *arXiv preprint arXiv:1609.04938*, 2017.
- [2] D. Gurgurov and A. Morshnev, “Image-to-latex converter for mathematical formulas and text,” *arXiv preprint arXiv:2408.04015*, 2024.
- [3] S. Peng, L. Gao, K. Yuan, and Z. Tang, “Image to LaTeX with graph neural network for mathematical formula recognition,” in *Document Analysis and Recognition – ICDAR 2021* (J. Lladós, D. Lopresti, and S. Uchida, eds.), vol. 12822 of *Lecture Notes in Computer Science*, (Cham), pp. 648–663, Springer, 2021.
- [4] M. Suzuki, F. Tamari, R. Fukuda, S. Uchida, and T. Kanahori, “Infty — an integrated ocr system for mathematical documents,” *International Journal on Document Analysis and Recognition*, 2003.
- [5] Y. Xie, H. Mouchère, F. Simistira Liwicki, S. Rakesh, R. Saini, M. Nakagawa, C. T. Nguyen, and T. Truong, “Icdar 2023 crohme: Competition on recognition of handwritten mathematical expressions,” in *Proceedings of the 17th International Conference on Document Analysis and Recognition (ICDAR)*, pp. 553–565, Springer, 2023.
- [6] S. Khan, “Im2latex-100k.” Kaggle Dataset, 2019.
- [7] Phymond, “Hme100k: A dataset for handwritten mathematical expression.” GitHub Repository, 2020.

A Exposure Bias During Joint Training

During early experiments, the model was trained jointly on the combined IM2LATEX-100K and HME100K datasets. Qualitative inspection revealed a generation failure where the decoder produced repeated token patterns unrelated to the input image. Although the outputs remained syntactically valid LaTeX, the generated expressions often collapsed toward common token sequences instead of matching the input structure.

For example, given the ground-truth expression:

$$[\{\bar{K}_a^-(p), \bar{K}_b^-(k)\}] = if_{abc} \bar{K}_c^-(p+k) - 2\pi^2 C p \delta_{ab} \delta_{p+k,0}$$

the model generated the following output:

$$\frac{K_N}{K} \frac{K_N}{K_N} \frac{K_N K_N K_N K_N}{K}$$

This indicates that the decoder learned LaTeX token distributions from teacher forcing but failed to sufficiently incorporate visual features from the encoder. The decoder learned sequence statistics faster than the encoder learned meaningful image representations, causing predictions to collapse toward frequent mathematical patterns.

This observation motivated the final staged training strategy, where the model first learned general LaTeX syntax and mathematical structure from IM2LaTeX-100K before fine-tuning visual representations on handwritten HME100K expressions.

B Overfitting Experiment

To verify that the CNN-Transformer architecture was capable of learning the image-to-LaTeX mapping, an overfitting experiment was performed using only 20 training samples and the same 20 validation samples. The model was trained for 100 epochs.

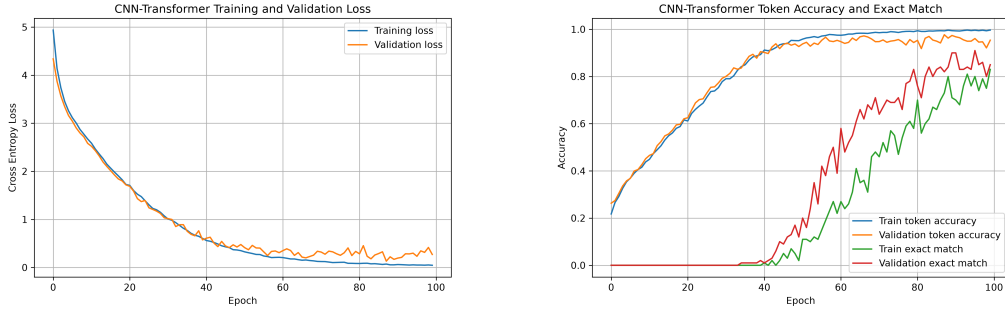


Figure 8: Training curves for the overfit test

The model successfully overfit the small dataset, with training loss decreasing from approximately 5 to near 0. Token accuracy increased from 0.21 to nearly 1.0, and exact match accuracy improved from 0% to approximately 85%. This confirms that the observed generation failures during early experiments were primarily related to training strategy and generalization rather than insufficient model capacity.